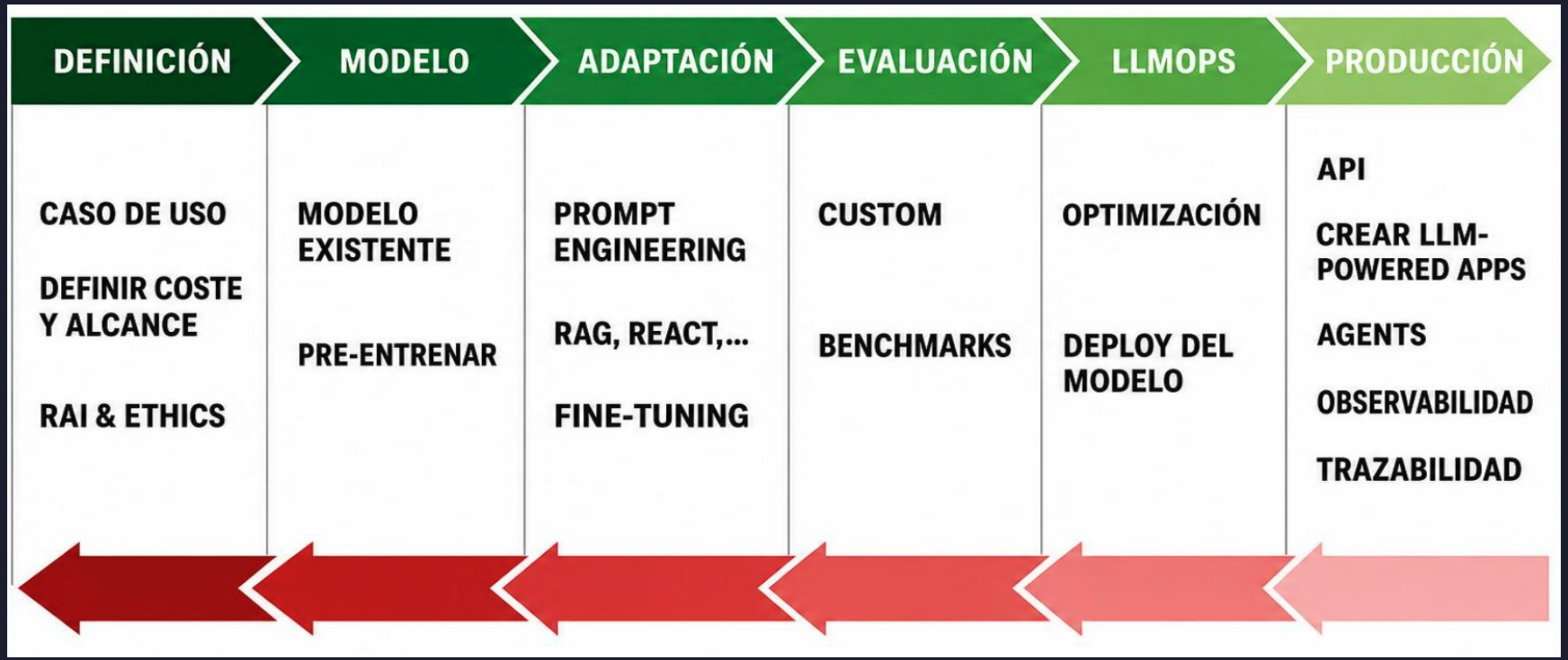
keep coding

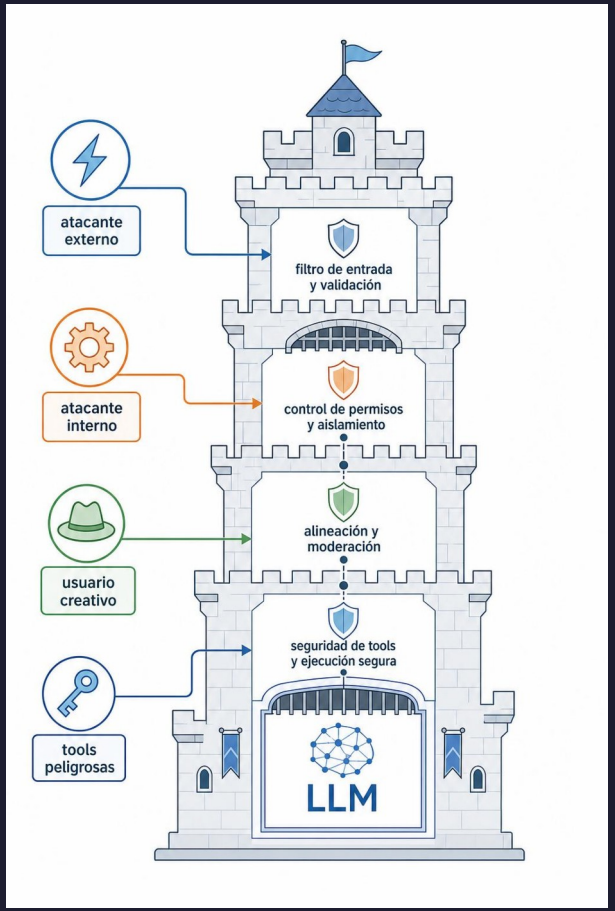
AI ENGINEERING

Safety


DEFINICIÓN
MODELO
ADAPTACIÓN
EVALUACIÓN
LLMOPS
PRODUCCIÓN
CASO DE USO
DEFINIR COSTE Y ALCANCE
RAI & ETHICS
MODELO EXISTENTE
PRE-ENTRENAR
PROMPT ENGINEERING
RAG, REACT,...
FINE-TUNING
CUSTOM
BENCHMARKS
OPTIMIZACIÓN
DEPLOY DEL MODELO
API
CREAR LLM-POWERED APPS
AGENTS
OBSERVABILIDAD
TRAZABILIDAD

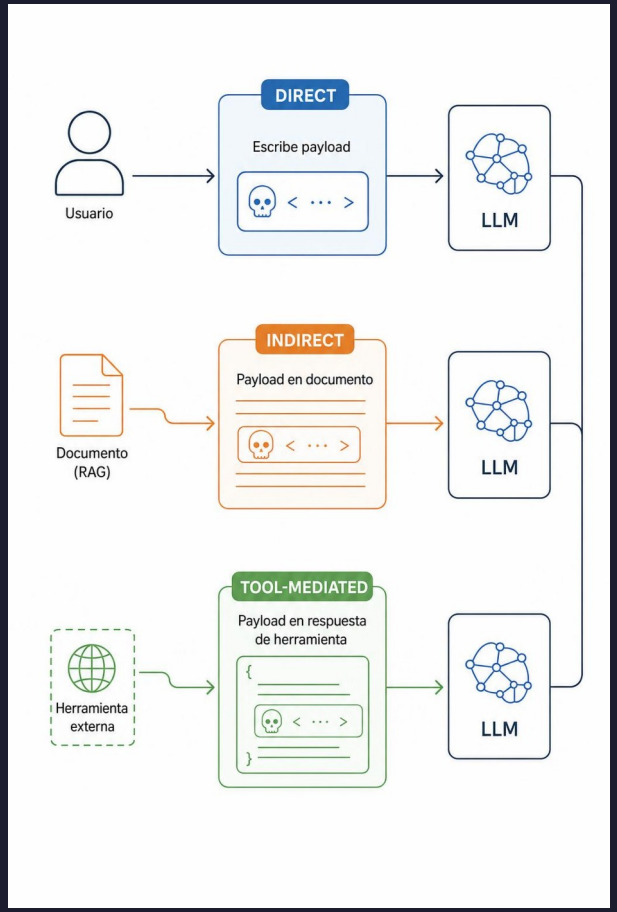
Threat model de una LLM app

- Atacante externo: jailbreaks y prompt injection
- Atacante interno: envenena el corpus del RAG
- Usuario creativo: payload smuggling
- El LLM como puerta a otros sistemas (tools)
- Defensa en capas obligatoria



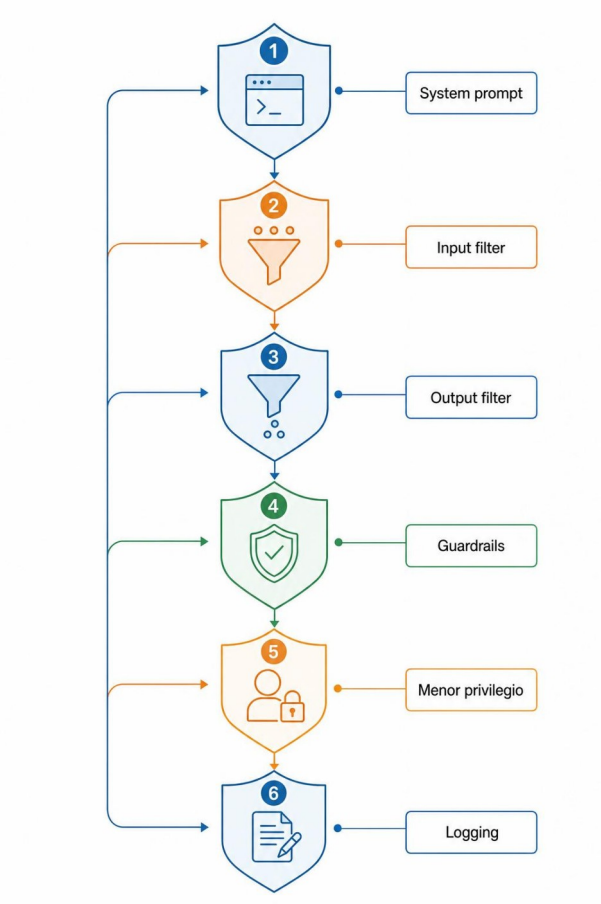
Prompt injection: la madre

- Directa: el usuario escribe 'ignora tus instrucciones'
- Indirecta: payload en datos de terceros (RAG, web)
- Vía tool: el payload llega en la respuesta de una tool
- No hay solución al 100 %
- Defensa en capas + monitorización



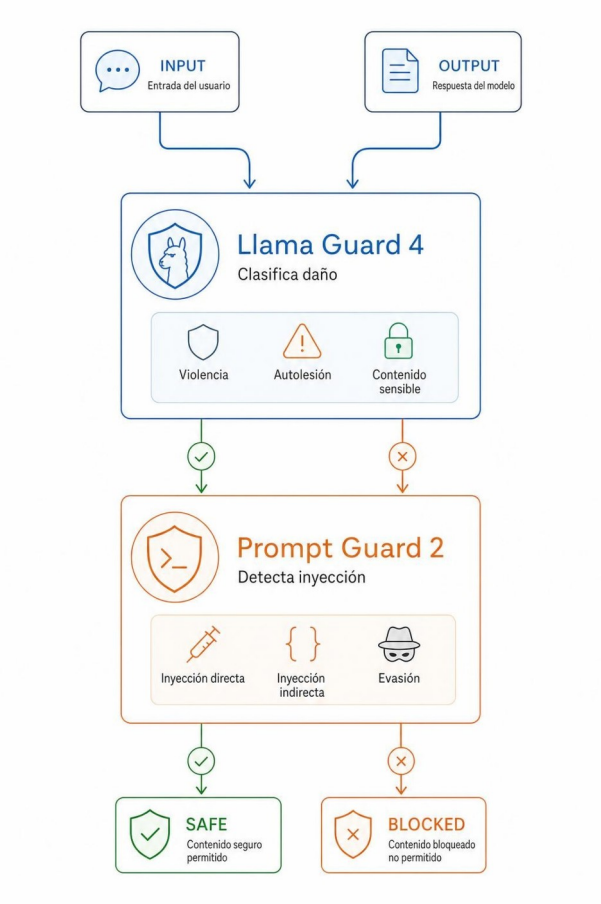
Defensa en capas y mínimo privilegio

- System prompt blindado con instrucciones inmutables
- Input filter: detecta el payload antes del LLM
- Output filter: corta fugas de secretos y PII
- Contexto de RAG y tools = datos, nunca instrucciones
- Mínimo privilegio: allowlist y confirmación humana en tools



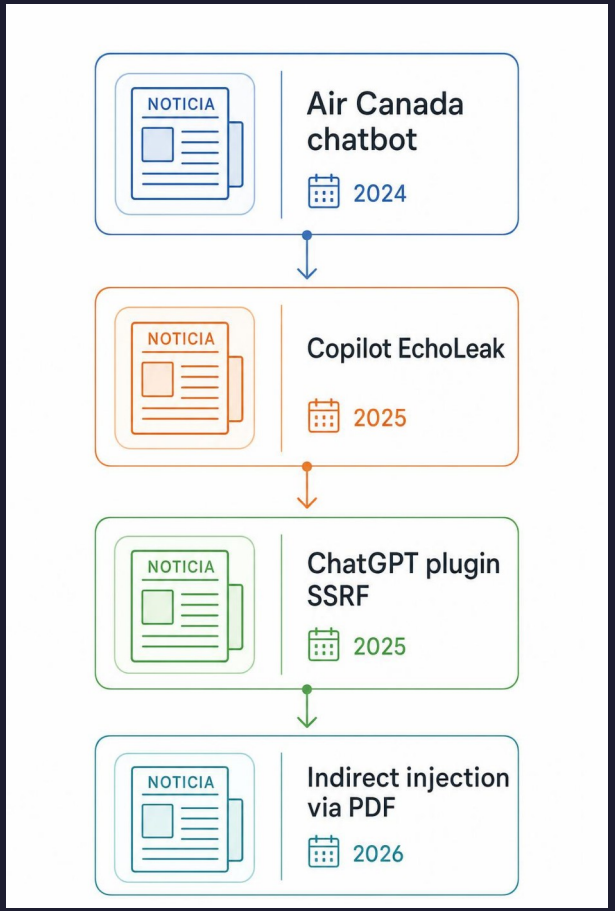
Guardrails: modelos y librerías

- Llama Guard 4: clasifica daño en entrada y salida
- Prompt Guard 2: detecta injection y jailbreaks
- Presidio: detecta y anonimiza PII sin LLM
- OpenAI Moderation: primer filtro gratuito
- Coste extra a cambio de menos falsos negativos



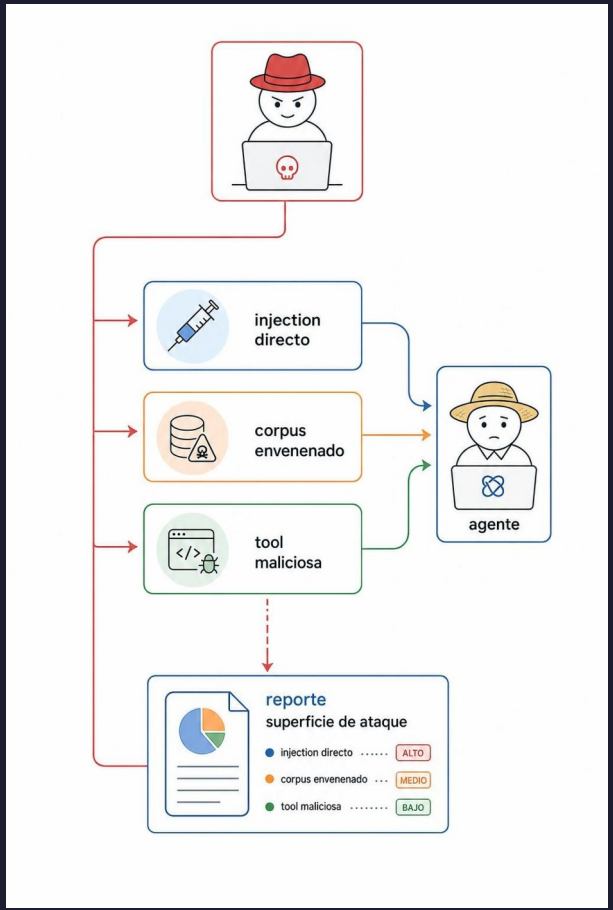
Casos reales 2025-2026

- Air Canada: el chatbot prometió un reembolso y el tribunal lo obligó
- Microsoft Copilot: exfiltración por un payload en un email (EchoLeak)
- Plugin de ChatGPT con SSRF
- La inyección indirecta vía PDFs es el vector más común
- Sin monitorización no detectas ataques en curso



Lab: red team a un agente RAG

- Atacamos un agente RAG con una tool de email
- Vector 1: prompt injection directa
- Vector 2: corpus envenenado (inyección indirecta)
- Vector 3: payload en una tool + defensas
- Lab: 01-red-team-agente-rag.ipynb




keep coding